

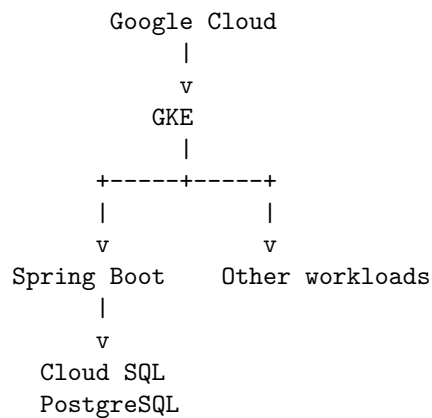
GKE, Load Balancer, Istio, Envoy, Cloud SQL Proxy, Liquibase, and Terraform

1. GKE

GKE = Google Kubernetes Engine.

GKE is Google's managed Kubernetes service. It is where the application containers run.

For our use case:



Kubernetes manages things such as:

- Pods
- Deployments
- Services
- Scaling
- Rolling deployments
- ConfigMaps and Secrets
- Namespaces
- Health checks

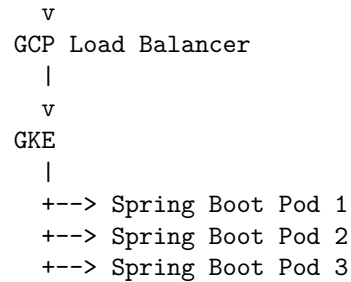
GKE provides the managed Kubernetes platform on Google Cloud.

2. Load Balancer

A **Load Balancer** handles incoming traffic and gets it to the application.

For example:

```
User
|
| HTTPS
```



A load balancer can provide:

- External IP
- HTTPS/TLS handling
- Health checks
- Distribution of traffic across application instances
- Routing traffic into GKE

Simple mental model

Load Balancer = gets external traffic to your application.

3. Istio

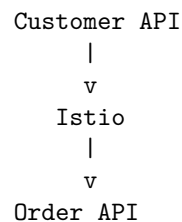
Istio is a service mesh.

It manages and controls communication between services running in Kubernetes.

Typical Istio capabilities include:

- Service-to-service mTLS
- Traffic routing
- Retries
- Timeouts
- Circuit breaking
- Canary deployments
- Service-to-service observability

For example:



Istio can control things such as:

Order API v1 -> 90%
Order API v2 -> 10%

Timeout -> 2 seconds
Retry -> 3 times
mTLS -> enabled

Important for our use case

We currently have **only one Spring Boot service**.

Therefore, Istio is probably **not necessary** unless we have a specific requirement such as:

- Service mesh
- mTLS
- Advanced traffic routing
- Canary routing
- Advanced service-to-service observability
- Retry/circuit-breaking policies at the mesh level

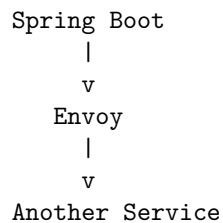
With one service, keeping the architecture simple is preferable.

4. Envoy

Envoy is a high-performance proxy.

Istio can use Envoy as the proxy that actually handles network traffic.

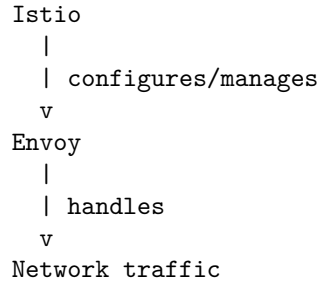
Conceptually:



Envoy can handle:

- Traffic routing
- mTLS
- Retries
- Timeouts
- Load balancing
- Telemetry

Relationship between Istio and Envoy



A useful mental model:

Istio = service mesh/control and traffic-management layer

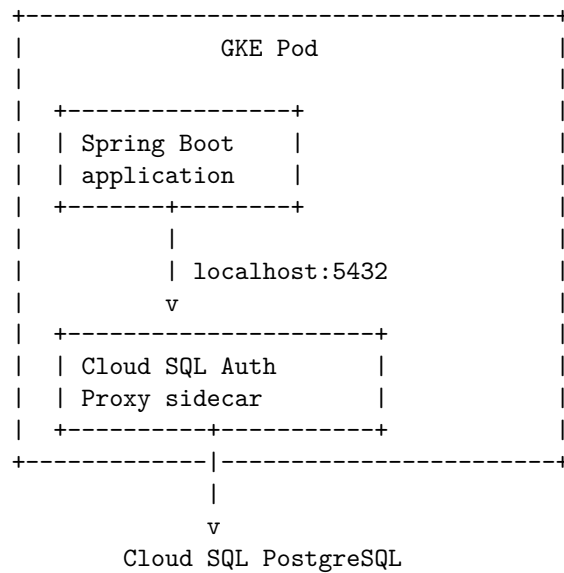
Envoy = proxy that handles network traffic

5. Cloud SQL Auth Proxy

The **Cloud SQL Auth Proxy** is different from Envoy.

It provides a secure connection path between the application and Cloud SQL.

If we use it as a sidecar:



The application can connect to:

localhost:5432

while the Cloud SQL Auth Proxy handles the connection to Cloud SQL.

Cloud SQL Proxy vs Envoy

Component	Main purpose
Cloud SQL Auth Proxy	Secure application-to-Cloud-SQL connection
Envoy	Service-to-service network proxy
Istio	Manages/configures the service mesh and Envoy
GCP Load Balancer	External traffic into the application

Important

Using a **Cloud SQL Proxy sidecar** does **NOT** mean that Istio is required.

You can have:

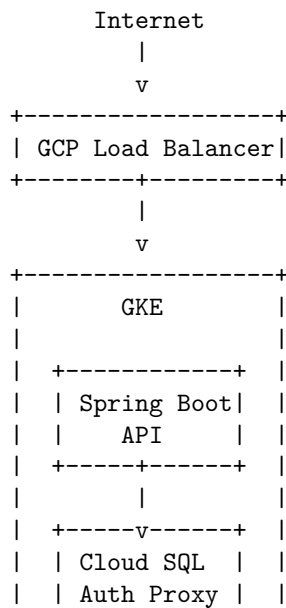
Spring Boot + Cloud SQL Proxy

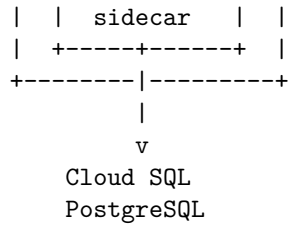
without:

Istio + Envoy

6. Recommended architecture for our current use case

Because we have only one Spring Boot service:





No Istio is required for this architecture.

7. Liquibase

Liquibase is a database schema/version management tool.

A simple mental model:

Liquibase is like Git for database schema changes.

Suppose version 1 has:

```

CREATE TABLE customer (
  id BIGINT PRIMARY KEY,
  name VARCHAR(100)
);
  
```

Later, we need an email column.

Liquibase can define:

```

databaseChangeLog:
- changeSet:
  id: 002
  author: developer
  changes:
  - addColumn:
    tableName: customer
    columns:
    - column:
      name: email
      type: varchar(255)
  
```

Liquibase executes the change and records it.

Liquibase maintains tracking tables such as:

```

DATABASECHANGELOG
DATABASECHANGELOGLOCK
  
```

This allows Liquibase to know which migrations have already been executed.

For example:

```
001 -> executed
002 -> executed
003 -> pending
```

When the deployment runs, Liquibase applies the pending migration.

8. Terraform

Terraform is Infrastructure as Code (IaC).

It allows infrastructure to be defined using configuration files instead of manually creating resources.

For our architecture, Terraform can create:

```
GCP
|
+-- VPC
+-- Subnets
+-- GKE
+-- Cloud SQL
+-- IAM
+-- Service Accounts
+-- Load Balancer-related infrastructure
```

Example:

```
resource "google_container_cluster" "gke" {
  name      = "my-gke-cluster"
  location = "us-west1"
}
```

Terraform manages the desired infrastructure state.

9. Terraform Providers

A Terraform provider is a plugin that allows Terraform to communicate with an external platform/API.

For example:

```
Terraform
|
+-- Google Provider -----> Google Cloud
|
```

```

+--- Kubernetes Provider --> Kubernetes API
|
+--- Helm Provider -----> Helm/Kubernetes
|
+--- GitHub Provider -----> GitHub API

```

For our GCP infrastructure, we commonly use the Google provider:

```

terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"
    }
  }
}

provider "google" {
  project = "my-gcp-project"
  region  = "us-west1"
}

```

The provider knows how to communicate with the Google Cloud APIs.

10. Terraform Resource

A **resource** represents an individual infrastructure object managed by Terraform.

Example:

```

resource "google_sql_database_instance" "postgres" {
  name             = "my-postgres"
  database_version = "POSTGRES_17"
  region           = "us-west1"
}

```

This represents one Cloud SQL database instance.

Another example:

```

resource "google_container_cluster" "gke" {
  name     = "my-gke"
  location = "us-west1"
}

```

This represents one GKE cluster.

Mental model

Resource = individual infrastructure building block

Examples:

```
google_container_cluster
google_sql_database_instance
google_compute_network
google_service_account
```

11. Terraform Module

A **module is a reusable collection of Terraform configuration**, usually containing multiple resources, variables, and outputs.

Instead of putting everything into one large Terraform configuration:

```
main.tf
|
+-- VPC
+-- GKE
+-- Cloud SQL
+-- IAM
+-- Subnets
```

we can organize it into reusable modules:

```
modules/
|
+-- network/
|   +-- main.tf
|   +-- variables.tf
|   +-- outputs.tf
|
+-- gke/
|   +-- main.tf
|   +-- variables.tf
|   +-- outputs.tf
|
+-- cloudsql/
|   +-- main.tf
|   +-- variables.tf
|   +-- outputs.tf
```

Then an environment can call the modules:

```

module "network" {
  source = "../../modules/network"
}

module "gke" {
  source = "../../modules/gke"

  cluster_name = "dev-gke"
}

module "cloudsql" {
  source = "../../modules/cloudsql"
}

```

A GKE module could contain multiple resources:

```

module "gke"
|
+-- google_container_cluster
+-- google_container_node_pool
+-- google_service_account
+-- IAM resources

```

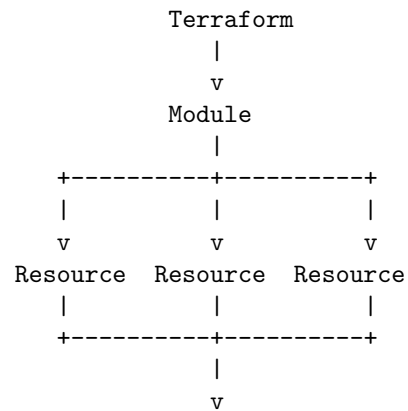
Mental model

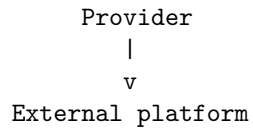
Resource = building block

Module = reusable collection of building blocks

12. Resource vs Module vs Provider

This is one of the most important Terraform distinctions.





More precisely, the provider is not literally “inside” a resource or module. Terraform uses the appropriate provider to perform the API operations required by resources.

Example

```

module "gke"
|
+-- google_container_cluster
+-- google_container_node_pool
+-- google_service_account
|
v
Google Provider
|
v
GCP
  
```

13. Terraform Lifecycle

The major Terraform workflow is:

```

Terraform Configuration
|
v
terraform init
|
v
Download Providers
|
v
terraform plan
|
v
Compare:
Desired State vs Current State
|
v
terraform apply
|
  
```

```
      v
Provider calls external APIs
      |
      v
Infrastructure created/updated
      |
      v
terraform.tfstate updated
```

terraform init

Downloads and initializes required providers and modules.

Example:

```
terraform init
```

Conceptually:

```
terraform init
      |
      v
Download Google Provider
      |
      v
Provider ready
```

terraform plan

Terraform calculates what needs to change.

Example:

Desired:

```
GKE cluster exists
```

Current:

```
GKE cluster does not exist
```

Plan:

```
+ Create GKE cluster
```

terraform apply

Terraform performs the planned changes.

```
Terraform
      |
      v
```

```
Google Provider
|
v
Google Cloud API
|
v
GKE created
```

Terraform then records the infrastructure state in:

```
terraform.tfstate
```

14. Terraform vs Liquibase

This distinction is very important.

Terraform manages infrastructure

```
Terraform
|
+-- GKE
+-- Cloud SQL instance
+-- VPC
+-- IAM
+-- Service Accounts
```

Liquibase manages database schema

```
Liquibase
|
+-- Tables
+-- Columns
+-- Indexes
+-- Constraints
+-- Views
+-- Stored procedures
```

So:

Terraform creates the database infrastructure.

Liquibase creates and evolves the schema inside the database.

For example:

```
Terraform
|
```

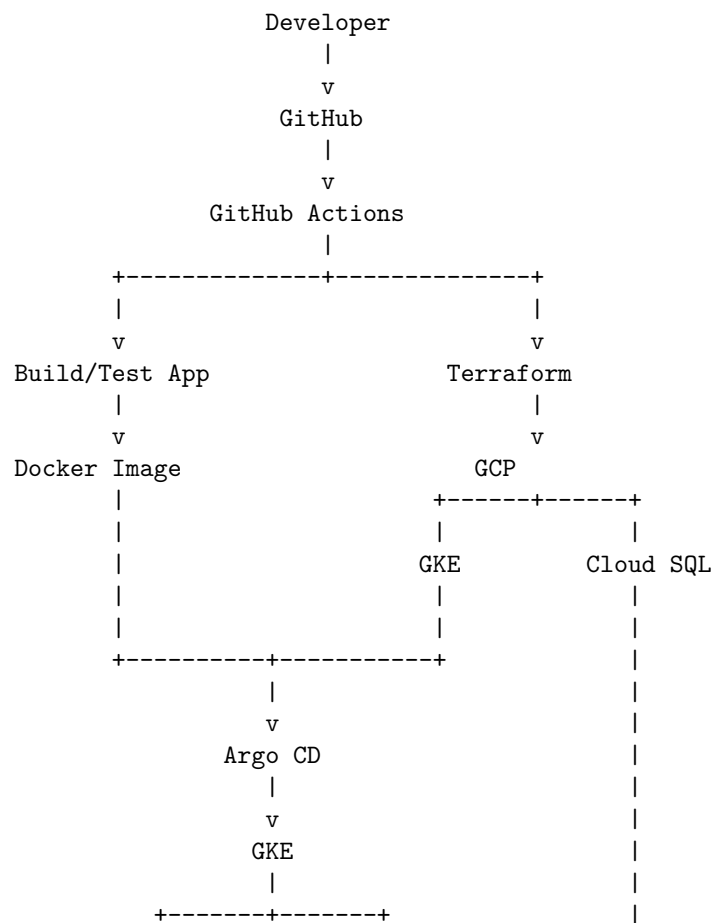
```

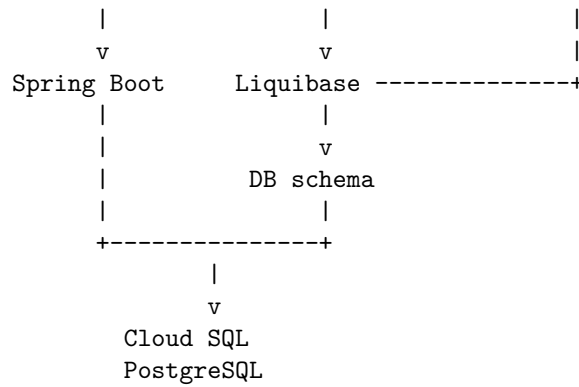
v
Create Cloud SQL PostgreSQL instance
|
v
Liquibase
|
+-- Create customer table
+-- Add email column
+-- Create orders table
+-- Create indexes

```

15. Full CI/CD architecture for our use case

Our technologies can fit together like this:





A cleaner deployment separation is:

Terraform

```

|
+--> Infrastructure
      +--> GKE
      +--> Cloud SQL
      +--> Networking
      +--> IAM
  
```

Argo CD

```

|
+--> Kubernetes application deployment
      +--> Spring Boot
      +--> Liquibase Job/migration
  
```

Spring Boot

```

|
+--> Cloud SQL Auth Proxy sidecar
      |
      v
      Cloud SQL
  
```

16. Ephemeral environment example

For a pull request, we might create:

PR #125

```

|
v
GitHub Actions
|
  
```

```

      v
Terraform
|
+--> GKE namespace/environment
|
+--> Cloud SQL instance/database
|
v
Argo CD
|
+--> Deploy Spring Boot
|
+--> Run Liquibase
|
v

```

Application ready

The new environment gets its database schema automatically:

New Cloud SQL database

```

      |
      v
Liquibase
|
+--> V1 schema
+--> V2 schema
+--> V3 schema
+--> V4 schema
|
v

```

Current schema

This makes the ephemeral environment reproducible and avoids manually configuring the database.

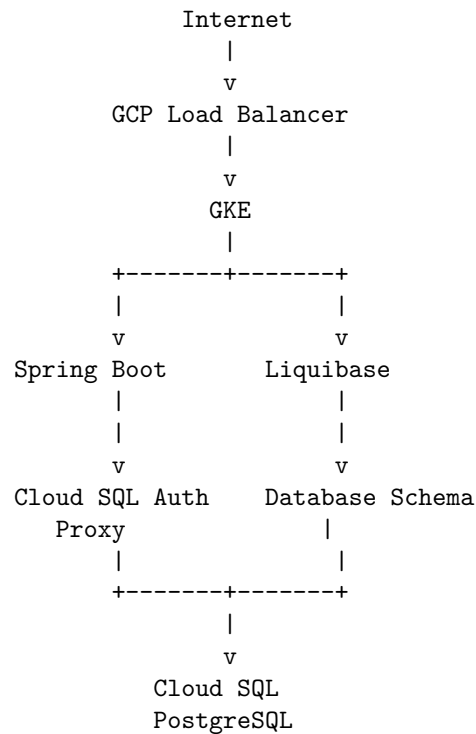
17. Final mental model

Remember these one-line definitions:

Technology	Simple definition
GKE	Runs your containers using managed Kubernetes
Load Balancer	Brings external traffic to your application

Technology	Simple definition
Istio	Manages service-to-service traffic in a service mesh
Envoy	Proxy that handles network traffic for the service mesh
Cloud SQL Auth Proxy	Secure proxy between application and Cloud SQL
Liquibase	Versions and applies database schema changes
Terraform	Defines and manages infrastructure as code
Provider	Terraform plugin that communicates with an external platform
Resource	Individual infrastructure object managed by Terraform
Module	Reusable collection of Terraform configuration/resources
Argo CD	Continuously deploys Kubernetes desired state from Git

Recommended architecture for our current single-service application



CI/CD:

GitHub

|

v

GitHub Actions

|

---> Terraform --> GCP infrastructure

|

v

Argo CD --> GKE application deployment

Istio/Envoy are intentionally left out because we currently have only one Spring Boot service and therefore don't have a strong service-mesh requirement.